A1.4 DISEÑO DEL MODELO DE DATOS

PROGRAMA EDUCATIVO

Ingeniería en Sistemas Computacionales

NOMBRE DEL PROYECTO

Automatización de mantenimiento preventivo con monitoreo permanente utilizando APIs y scripts

EMPRESA

Infraestructura Digital Predictiva Bitlab, S.A.P.I. de C.V.

PRESENTA

Axel Gabriel Ramírez Ortiz
Matrícula: 122043202 · Grupo: S206

ASESOR UPQ:

Ana Laura Lira Cortes

CO-ASESOR UPQ:

Ivan Peredo Valderrama

ASESORA EMPRESARIAL:

M.T.A. Sandra Lucía de la Fuente Bermúdez

Querétaro, Qro., 29 de octubre del 2026

ÍNDICE

1. OBJETIVO DE LA ACTIVIDAD

2. DESARROLLO DE LA ACTIVIDAD

- Principios del diseño
- Diagrama entidad-relación
- Modelo relacional de las tablas implementadas
- Tablas que se crearán después
- Índices
- Reglas de seguridad por registro

3. RESULTADOS DE LA ACTIVIDAD

4. PENDIENTES Y SIGUIENTES PASOS

REFERENCIAS

1. OBJETIVO DE LA ACTIVIDAD

En esta actividad diseñé cómo se va a guardar la información del módulo: el catálogo de comandos, cada ejecución que se lance, lo que responde cada equipo y los reportes del agente. También definí los índices que harán rápidas las consultas del panel y las reglas que mantienen separada la información de cada cliente.

Responde a la tercera pregunta del proyecto, ¿cómo debe organizarse la base de datos para guardar las ejecuciones y alertas, mantener separada la información de cada cliente y responder rápido al panel?, y prepara la hipótesis H3, que plantea que las reglas de separación no harán más lento el panel si existen los índices adecuados.

Tabla 1. Datos de la actividad.

Concepto	Descripción
Clave de la actividad	A1.4
Fase del protocolo	F1. Requerimientos y arquitectura
Semana	S3 (21 al 25 de septiembre de 2026)
Horas planeadas	24 horas
Producto	Modelo de datos documentado
Entregable al que pertenece	E1. Documento de requerimientos y arquitectura (sub-entregable E1.4)

Fuente: Protocolo de estadía profesional, Bitlab-IT (2026).

2. DESARROLLO DE LA ACTIVIDAD

Principios del diseño

Antes de dibujar cualquier tabla había que tomar una decisión de fondo. El módulo es algo **nuevo** que se suma a una plataforma que ya está funcionando con clientes reales, y esa plataforma no se puede modificar. Por eso decidí que el módulo tuviera **sus propias tablas** y que de la plataforma actual solo **leyera** lo que necesita, como los equipos, las alertas y los clientes. Si algo sale mal en el módulo, lo que ya existe sigue igual.

A partir de ahí fijé el resto de las reglas. Se usa Supabase sobre PostgreSQL, que es lo mismo que usa la plataforma actual (Supabase, 2024). Cada cambio a la base se

hace con una migración, un archivo con fecha que se guarda en el repositorio y que se puede repetir y revisar. Los nombres van en español para que cualquiera del equipo los entienda. Las reglas de seguridad por registro se activan en todas las tablas desde el primer día. Y el agente nunca tiene la llave maestra de la base: solo puede llamar a unas funciones especiales que revisan su clave y le dejan trabajar únicamente con su propio equipo.

Una parte del diseño ya está hecha realidad. La primera migración del proyecto, 20261028000000_demo_terminal.sql, crea cinco tablas, sus reglas de seguridad, las funciones del agente y el catálogo inicial, y ya la probé en una base PostgreSQL local. Las demás tablas que se describen aquí se crearán en las siguientes migraciones, dentro del entregable E4.

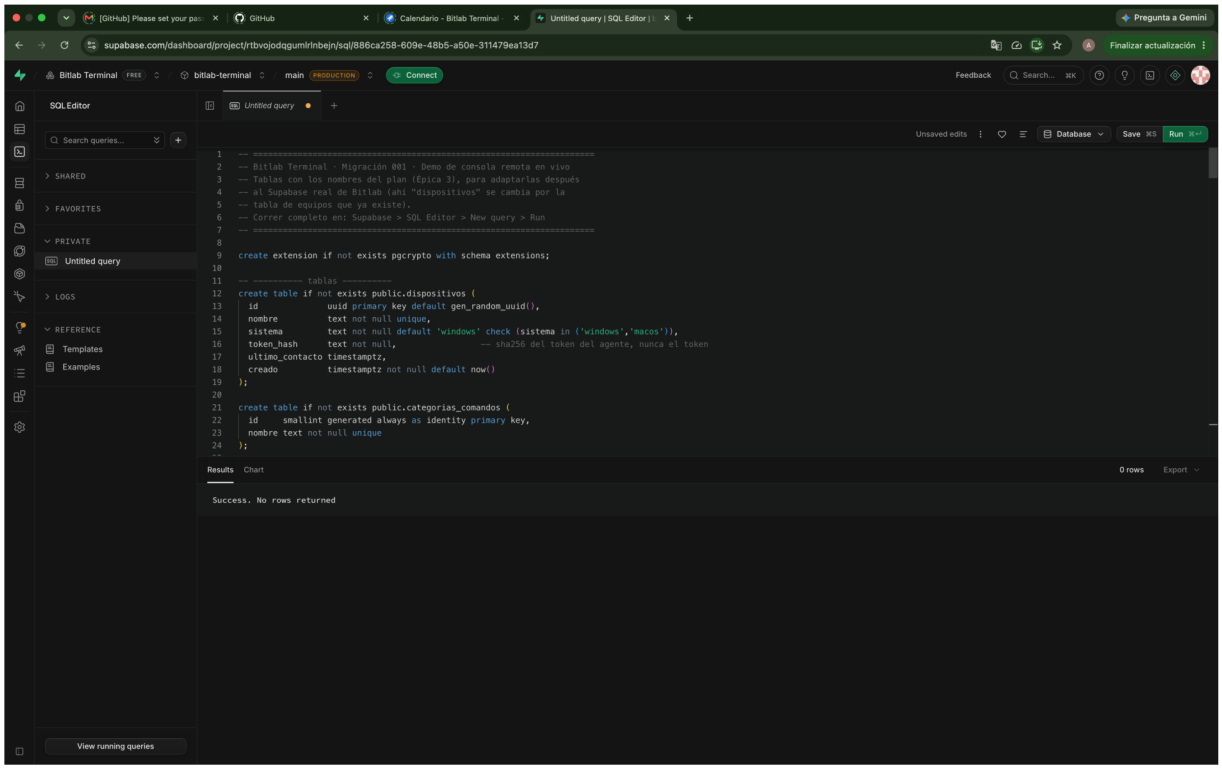


Figura 1. Primera migración ejecutada en Supabase. *Fuente: Elaboración propia.*

https://github.com/axelbitlab/bitlab/blob/main/supabase/migrations/20261028000000_demo_terminal.sql

Diagrama entidad-relación

El diagrama de la Figura 2 muestra las tablas del módulo y cómo se relacionan. Las que están en mayúsculas pertenecen a la plataforma actual y solo se leen; el resto son las tablas nuevas del módulo.

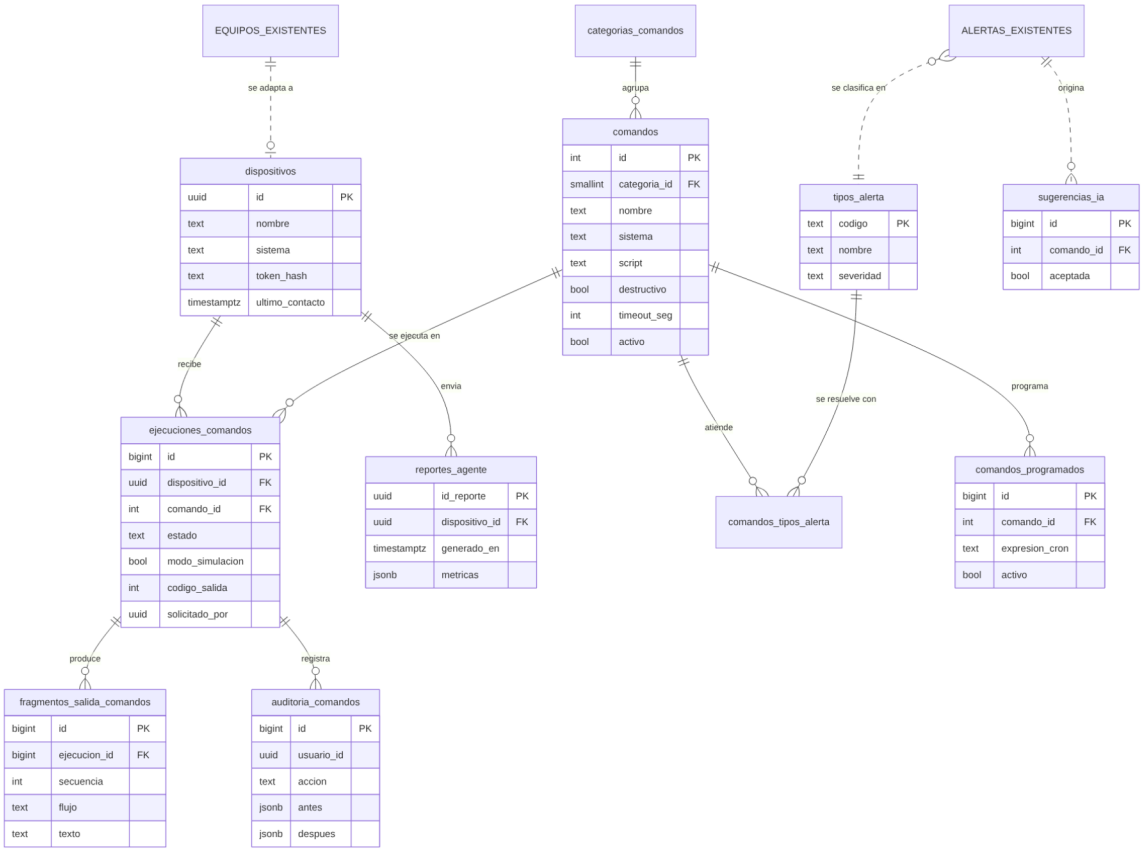


Figura 2. Diagrama entidad-relación del módulo. *Fuente: Elaboración propia.*

La relación central es la de comandos con ejecuciones: un comando del catálogo se puede ejecutar muchas veces en muchos equipos, y cada ejecución produce muchas líneas de salida. Alrededor de eso están los tipos de alerta, que se ligan con los comandos que las atienden, y las tablas que se agregarán después para los reportes, los comandos programados, la auditoría y las sugerencias de la inteligencia artificial.

Modelo relacional de las tablas implementadas

El diagrama entidad-relación muestra la idea general, pero para construir la base hace falta el detalle de cada tabla: sus columnas, el tipo de dato, cuál es la llave primaria,

cuáles apuntan a otras tablas y qué restricciones tiene cada campo. Las Tablas 2 a 6 muestran el modelo relacional de las cinco tablas que ya crea la primera migración.

Tabla 2. Modelo relacional de la tabla dispositivos.

Columna	Tipo	Llave	Nulo	Restricción
id	uuid	PK	No	Se genera solo con gen_random_uuid().
nombre	text		No	Único. Es el nombre del equipo.
sistema	text		No	Solo windows o macos. Por defecto windows.
token_hash	text		No	Huella SHA-256 de la clave del agente; nunca la clave real.
ultimo_contacto	timestamptz		Sí	Se actualiza cada vez que el agente pregunta por comandos.
creado	timestamptz		No	Fecha de alta. Por defecto now().

Fuente: Elaboración propia.

Tabla 3. Modelo relacional de la tabla categorias_comandos.

Columna	Tipo	Llave	Nulo	Restricción
id	smallint	PK	No	Identidad generada automáticamente.
nombre	text		No	Único. Diagnóstico, Red o Procesos.

Fuente: Elaboración propia.

Tabla 4. Modelo relacional de la tabla comandos.

Columna	Tipo	Llave	Nulo	Restricción
id	int	PK	No	Identidad generada automáticamente.
categoria_id	smallint	FK → categorias_comandos.id	Sí	Categoría del comando.
nombre	text		No	Único junto con sistema.
sistema	text		No	Solo windows o macos.
script	text		No	Texto exacto que ejecuta el agente.
destructivo	boolean		No	Indica si modifica el equipo. Por defecto false.
timeout_seg	int		No	Tiempo máximo de ejecución. Por defecto 60.
activo	boolean		No	Solo los activos aparecen en el panel. Por defecto true.

Fuente: Elaboración propia.

Tabla 5. Modelo relacional de la tabla ejecuciones_comandos.

Columna	Tipo	Llave	Nulo	Restricción
id	bigint	PK	No	Identidad generada automáticamente.
dispositivo_id	uuid	FK → dispositivos.id	No	Equipo donde se ejecuta.
comando_id	int	FK → comandos.id	No	Siempre un comando del catálogo.
estado	text		No	pendiente, ejecutando, exitosa, fallida, cancelada o tiempo_agotado. Por defecto pendiente.
modo_simulacion	boolean		No	Si es true, el agente solo describe lo que haría.
codigo_salida	int		Sí	Código con el que terminó el proceso.
solicitado_por	uuid	FK → auth.users.id	Sí	Técnico que lanzó la ejecución. Por defecto auth.uid().
creado, iniciado, terminado	timestamptz		creado no; los otros sí	Fechas de cada momento; sirven para medir tiempos.

Fuente: Elaboración propia.

Tabla 6. Modelo relacional de la tabla fragmentos_salida_comandos.

Columna	Tipo	Llave	Nulo	Restricción
id	bigint	PK	No	Identidad generada automáticamente.
ejecucion_id	bigint	FK → ejecuciones_comandos.id	No	Se borra en cascada con su ejecución.
secuencia	int		No	Orden de la línea. Única junto con ejecucion_id.
flujo	text		No	stdout, stderr o sistema. Por defecto stdout.
texto	text		No	Hasta 4000 caracteres por línea.
creado	timestamptz		No	Momento en que llegó la línea.

Fuente: Elaboración propia.

Hay tres detalles de este modelo que vale la pena explicar. El primero es que la clave del equipo nunca se guarda tal cual, solo su huella; así, aunque alguien viera la tabla,

no podría hacerse pasar por un equipo. El segundo es que cada ejecución sigue un camino de estados con su fecha, lo que permite medir después cuánto tardó en llegar al equipo y cuánto duró. El tercero es que la combinación de ejecución y número de línea no se puede repetir: si el agente reenvía datos por una falla de red, la consola no muestra líneas duplicadas.

Tablas que se crearán después

Para completar el módulo diseñé ocho tablas más, que se irán creando en los entregables E4, E5 y E6.

Tabla 7. Tablas por crear en las siguientes migraciones.

Tabla	Para qué sirve	Columnas principales	Entregable
tipos_alerta	Los 11 tipos de alerta de la actividad A1.3.	codigo (PK), nombre, severidad, condicion	E4
comandos_tipos_alerta	Liga cada alerta con los comandos que la atienden; con ella se arma el menú del panel.	comando_id (FK), tipo_alerta (FK), orden	E4
auditoria_comandos	Registro automático de quién cambió qué y cuándo.	id (PK), usuario_id, accion, entidad, antes, despues, creado	E4
reportes_agente	Los reportes periódicos de cada equipo (A1.5).	id_reporte (PK), dispositivo_id (FK), generado_en, metricas (jsonb)	E5
comandos_programados	Comandos que corren solos en un horario.	id (PK), comando_id (FK), dispositivo_id (FK), expresion_cron, activo	E5
sugerencias_ia	Lo que sugirió la IA y si el técnico lo aceptó.	id (PK), alerta_id, comando_id (FK), justificacion, aceptada	E6
comandos_guardados_usuario	Los comandos frecuentes de cada técnico.	id (PK), usuario_id, comando_id (FK), etiqueta, orden	E6
lotes_ejecuciones	Agrupa las ejecuciones lanzadas a varios equipos a la vez.	id (PK), comando_id (FK), total, exitosas, fallidas	E6

Fuente: Elaboración propia.

Todas las tablas que guardan información de un cliente llevarán además una columna que indica a qué cliente pertenece. Esa columna es la que usan las reglas de seguridad para separar la información.

Índices

Un índice funciona como el índice de un libro: ayuda a encontrar la información sin revisar toda la tabla. No quise poner índices en todo, porque cada uno hace un poco más lenta la escritura; solo los puse donde hay una consulta frecuente que lo justifica.

Tabla 8. Índices del módulo y la consulta que justifica cada uno.

Índice	Consulta que lo justifica	Estado
Equipo, estado y fecha en ejecuciones	El agente pregunta cada pocos segundos por la ejecución pendiente más antigua de su equipo.	Implementado
Ejecución y número de línea en la salida	Mostrar la salida en orden y evitar líneas duplicadas.	Implementado
Nombre y sistema en comandos	Evitar comandos repetidos para un mismo sistema.	Implementado
Cliente y fecha en ejecuciones	Historial del panel filtrado por cliente.	Por crear
Equipo y fecha en reportes	Último reporte de cada equipo y sus tendencias.	Por crear
Tipo de alerta y orden en comandos	Menú de comandos al seleccionar una alerta.	Por crear

Fuente: Elaboración propia.

Para comprobar la hipótesis H3, en el entregable E4 voy a medir el tiempo de estas consultas con las reglas de seguridad activadas y sin ellas.

Reglas de seguridad por registro

Como varios clientes comparten la misma base de datos, cada uno debe ver solo lo suyo. Para eso PostgreSQL tiene las reglas de seguridad por registro, que revisan fila por fila si un usuario puede ver o cambiar un dato (The PostgreSQL Global Development Group, 2024). La primera migración ya aplica varias:

- Las reglas están activadas en las cinco tablas.
- Los técnicos con sesión pueden ver los equipos, el catálogo, las ejecuciones y la salida, y pueden lanzar ejecuciones.
- La huella de la clave de cada equipo no se puede consultar desde el panel.
- Una ejecución solo se acepta si la pide el mismo usuario, empieza como pendiente y usa un comando activo del catálogo. En otras palabras, desde el panel no se puede ejecutar nada que no esté en el catálogo.

- Las funciones del agente revisan su clave y solo le dejan trabajar con las ejecuciones de su propio equipo.
- El alta de un equipo solo se hace desde el editor de la base de datos y la clave se muestra una sola vez. Si se vuelve a registrar el equipo, se genera una clave nueva y la anterior deja de servir.

En la demostración, las reglas de lectura dejan ver todos los registros, porque por ahora solo hay un usuario de prueba. Para producción cada regla se va a filtrar por cliente, con una forma parecida a esta:

```
create policy ejecuciones_por_organizacion on public.ejecuciones_comandos
 for select to authenticated
 using (organizacion_id in (select public.organizaciones_del_usuario()));
```

La función que dice a qué clientes pertenece cada usuario depende de cómo la plataforma actual relaciona a sus usuarios con sus clientes, y eso lo podré ver hasta tener acceso. El protocolo pide probar estas reglas con al menos dos clientes de prueba: un técnico del cliente A no debe ver ni ejecutar nada en los equipos del cliente B.

3. RESULTADOS DE LA ACTIVIDAD

Al terminar esta actividad quedó diseñada la base de datos completa del módulo, como se resume en la Tabla 9.

Tabla 9. Resumen del modelo de datos.

Elemento	Resultado
Diagrama entidad-relación	Con las tablas del módulo y su relación con las de la plataforma actual.
Modelo relacional	Detallado para las 5 tablas implementadas y con columnas principales para las 8 por crear.
Tablas implementadas	5, en la primera migración, probada en PostgreSQL local.
Índices	3 implementados y 3 por crear, cada uno justificado por una consulta.
Reglas de seguridad	Activadas en todas las tablas; falta filtrarlas por cliente para producción.
Tiempo real	Las tablas de ejecuciones y de salida mandan sus cambios al panel al momento.

Fuente: Elaboración propia.

También propuse cuánto tiempo guardar la información, para validarlo con la asesora: la salida de los comandos, 90 días; las ejecuciones y la auditoría, un año; y los reportes del agente, 30 días, dejando un resumen diario.

4. PENDIENTES Y SIGUIENTES PASOS

- Conocer la estructura de las tablas actuales de equipos, alertas, clientes y usuarios, cuando tenga acceso a la plataforma.
- Definir la columna de cliente que usarán las reglas de seguridad.
- Validar con la asesora cuánto tiempo se guardará la información.
- Contar con un entorno de Supabase de desarrollo separado del de producción, como pide el protocolo.

Con la base de datos diseñada, sigue la actividad A1.5, donde defino cómo el agente envía su información a la plataforma.

REFERENCIAS

Bitlab-IT. (2026). Protocolo de estadía profesional (versión 1.0).

Supabase. (2024). Supabase documentation: Database migrations and Row Level Security. https://supabase.com/docs

The PostgreSQL Global Development Group. (2024). PostgreSQL documentation: Row security policies. https://www.postgresql.org/docs/current/ddl-rowsecurity.html